

Practical-Week1

1. Accept a Linux command as input.
2. Create a child process using fork().
3. Execute the command in the child process using an appropriate exec() system call.
4. Allow the parent process to wait for the child using wait ().
5. Display the Process ID (PID) of both parent and child processes.

```
sanjay_dundra@Sanjay:~$ nano process.c
sanjay_dundra@Sanjay:~$ gcc process.c -o process
sanjay_dundra@Sanjay:~$ ./process
Enter a Linux command: ls
Parent Process PID: 627
Child Process PID: 628
Executing command: ls
OSSP      assets      hello      process.c
OS_Lab    command_executor.c  process    shell_project
Parent process: Child execution completed.
sanjay_dundra@Sanjay:~$ |
```

```
sanjay_dundra@Sanjay:~$ ./process
Enter a Linux command: date
Parent Process PID: 675
Child Process PID: 678
Executing command: date
Fri Aug 14 09:53:31 UTC 2026
Parent process: Child execution completed.
sanjay_dundra@Sanjay:~$ |
```

Report: I ran process.c with two commands: ls (parent PID 627, child PID 628) and date (parent PID 675, child PID 678). Both executed correctly with distinct, valid PIDs, and

the parent reported completion after each. This confirms the program correctly forks and execs arbitrary user-specified commands.

2. Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
sanjay_dundra@Sanjay:~$ uname -a
Linux Sanjay 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
```

```
sanjay_dundra@Sanjay:~$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0      0 356.9M 1 disk
sdb   8:16     0 159.4M 1 disk
sdc   8:32     0 2G      0 disk [SWAP]
sdd   8:48     0 1T      0 disk /mnt/wslg/distro
```

```
sanjay_dundra@Sanjay:~$ ps
  PID TTY          TIME CMD
  403 pts/0        00:00:00 bash
  754 pts/0        00:00:00 ps
```

Report: I checked the system using `uname -a`, `lsblk`, and `ps`. `uname -a` confirmed the kernel (WSL2 Linux). `lsblk` listed disks including root (`sdd`, 1T) and swap (`sdc`, 2G). `ps` showed the active session's processes: `bash` (PID 403) and `ps` (PID 754). This confirms the environment inspection commands correctly reflect the running system.

sanjay_dundra@Sanjay:~\$ top

top - 09:57:03 up 8 min, 1 user, load average: 0.01, 0.02, 0.00

Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie

%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 99.9 id, 0.1 wa, 0.0 hi, 0.0 si,

MiB Mem : 7782.6 total, 6322.9 free, 539.9 used, 1072.3 buff/cache

MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7242.8 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+
1	root	20	0	24336	15456	11628	S	0.0	0.2	0:00.86
2	root	20	0	3180	2200	2072	S	0.0	0.0	0:00.00
7	root	20	0	3180	2040	1940	S	0.0	0.0	0:00.00
47	root	19	-1	50400	16820	15516	S	0.0	0.2	0:00.21
83	systemd+	20	0	22416	14476	11980	S	0.0	0.2	0:00.11
92	root	20	0	35368	12252	9164	S	0.0	0.2	0:00.52
111	root	20	0	2888	1952	1820	S	0.0	0.0	0:00.03
112	root	20	0	4472	3148	2900	S	0.0	0.0	0:00.01
113	message+	20	0	8796	5508	4832	S	0.0	0.1	0:00.04
161	root	20	0	44096	29704	14564	S	0.0	0.4	0:00.20
174	root	20	0	18436	9412	8168	S	0.0	0.1	0:00.01
179	root	20	0	1792360	15396	12552	S	0.0	0.2	0:00.08
191	syslog	20	0	220548	5916	4632	S	0.0	0.1	0:00.06
283	_chrony	20	0	23924	11368	7132	S	0.0	0.1	0:00.13
301	_chrony	20	0	12096	2468	1824	S	0.0	0.0	0:00.02
347	root	20	0	123460	32712	18096	S	0.0	0.4	0:00.20
350	root	20	0	5492	2728	2540	S	0.0	0.0	0:00.00
401	root	20	0	3188	1104	980	S	0.0	0.0	0:00.00
402	root	20	0	3204	1244	1108	S	0.0	0.0	0:00.02
403	ajithmut+	20	0	6268	5528	3856	S	0.0	0.1	0:00.06
404	root	20	0	8644	5528	4688	S	0.0	0.1	0:00.00
448	ajithmut+	20	0	22516	13432	10992	S	0.0	0.2	0:00.17
450	ajithmut+	20	0	22920	4092	2088	S	0.0	0.1	0:00.00
476	ajithmut+	20	0	6136	5432	3864	S	0.0	0.1	0:00.03
764	ajithmut+	20	0	8172	6000	3888	R	0.0	0.1	0:00.03